

For this football club system, should `Player` be abstract with `calculateMonthlySalary()` left to `StarPlayer`/`RegularPlayer` to override, or should `Player` be one concrete class with a 'type' field and if/else salary logic inside it?

Weighed OOP design patterns against pedagogical intent and codebase consistency >

This is a genuine architecture decision, so here's the real tradeoff — not just "which is more OOP-correct," but which actually fits *your* system better.

Option A: Abstract `Player` + subclass overrides (your current design)

Pros:

- Adding a third player type (say, `VeteranPlayer`) means writing one new class — zero changes to `PlayerList`, `SalaryList`, or anywhere else. This is the Open/Closed Principle in action: open for extension, closed for modification.
- The compiler *forces* every subclass to implement `calculateMonthlySalary()` — you literally cannot forget to define behavior for a new type. With an if/else version, forgetting a branch just silently falls through to some default (or a bug).
- No risk of a "type" field getting out of sync with actual behavior (e.g., someone sets `type = "Star"` but the salary math wasn't updated to match).
- Already proven out in your codebase — `StarPlayer` / `RegularPlayer` don't *just* differ in salary rate, they also differ in `displayInfo()` (different benefit text, different bonus-rate label). That divergence is real evidence the type already drives multiple behaviors, not just one number.

Cons:

- More files/classes for what's currently just a single percentage difference.
- Harder to change a player's type after creation (you'd have to delete and recreate the object) — abstract subclassing fixes type at construction.

Option B: One concrete `Player` class, `String` type field, if/else inside

`calculateMonthlySalary()`

Pros:

- Fewer files. Can flip a player's type by just reassigning the field.
- Feels simpler for *only two* cases with a single varying number.

Cons:

- That if/else doesn't stay contained to one method. Since your `displayInfo()` already diverges by type too, you'd need a *second* if/else there, and a *third* anywhere else type-specific text/logic shows up. Every new type means hunting down every branch point across the class and adding a case — easy to miss one.
- Typos in the type string (`"star"` vs `"Star"`) are a runtime bug with no compiler warning, versus a missing `@Override` method which is a compile error.
- Loses the actual pedagogical point — abstraction/polymorphism is one of your DTC components, and this is precisely the textbook case where overriding beats branching.

My recommendation

Keep `Player` abstract with `StarPlayer` / `RegularPlayer` overriding `calculateMonthlySalary()`. Your own codebase already shows the type affects more than one method (salary calc *and* display text), which is exactly the situation where if/else duplication compounds and overriding pays off. The only scenario where I'd switch to the type-field approach is if you needed to *change*  *ayer's type at runtime* as a business rule (e.g., "Regular players get promoted to Star after good performance reviews") — that's a real

My recommendation

Keep `Player` abstract with `StarPlayer` / `RegularPlayer` overriding `calculateMonthlySalary()`. Your own codebase already shows the type affects more than one method (salary calc *and* display text), which is exactly the situation where if/else duplication compounds and overriding pays off. The only scenario where I'd switch to the type-field approach is if you needed to *change a player's type at runtime* as a business rule (e.g., "Regular players get promoted to Star after good performance reviews") — that's a real reason to prefer mutable state over fixed class identity. Does your assignment spec mention anything like in-place type changes, or is type fixed once a player is created?